INITIATION PYTHON · JOUR 2 SUR 5 · MARDI 3 NOVEMBRE

# Mardi · Boucles, listes, chaînes, fonctions

Hier, vos programmes s'exécutaient de haut en bas, une seule fois. Aujourd'hui, on répète (les boucles), on range plusieurs valeurs dans une seule variable (les listes), on découpe du texte (les chaînes), et on donne un nom à un morceau de code pour le réutiliser (les fonctions). Avec ça, vous aurez tout ce qu'il faut pour le pendu de demain.

## 1. La boucle `while`

`while` répète un bloc **tant que** la condition est vraie.

rebours.py

```
n = int(input("Compter à rebours depuis : "))

while n > 0:
    print(n)
    n = n - 1

print("Décollage !")
```

Dans le terminal

```
$ python3 rebours.py
Compter à rebours depuis : 5
5
4
3
2
1
Décollage !
```

À chaque tour, Python revérifie `n > 0`. Quand `n` vaut 0, la condition devient fausse et le programme continue après le bloc. `n = n - 1` s'écrit aussi `n -= 1` (de même `+=`, `*=`).

> **Piège.** Oublier le `n = n - 1` : la condition reste vraie pour toujours, le programme tourne sans fin. `Ctrl+C` dans le terminal l'arrête.

`while` est parfait quand on ne sait pas d'avance combien de tours il faudra. Par exemple, redemander tant que la saisie n'est pas valable :

saisie_note.py

```
note = int(input("Note sur 20 : "))
while note < 0 or note > 20:
    print("La note doit être entre 0 et 20.")
    note = int(input("Note sur 20 : "))
print(f"Note enregistrée : {note}")
```

On tape 25, puis -3, puis 14

```
$ python3 saisie_note.py
Note sur 20 : 25
La note doit être entre 0 et 20.
Note sur 20 : -3
La note doit être entre 0 et 20.
Note sur 20 : 14
Note enregistrée : 14
```

## 2. La boucle `for` et `range`

Quand on sait combien de tours faire, on utilise `for` :

table.py

```python
n = int(input("Table de : "))
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Dans le terminal

```
$ python3 table.py
Table de : 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

`range(1, 11)` produit les nombres de 1 à 10 : **le deuxième nombre est exclu**. À chaque tour, `i` prend la valeur suivante.

Dans l'interpréteur Python

```
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 20, 5))
[0, 5, 10, 15]
>>> list(range(10, 0, -1))
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

| Écriture | Produit | En bash |
|---|---|---|
| `range(5)` | 0, 1, 2, 3, 4 | `for (( i = 0; i < 5; i++ ))` |
| `range(1, 11)` | 1 à 10 | `for i in {1..10}` |
| `range(0, 20, 5)` | 0, 5, 10, 15 | `for (( i = 0; i < 20; i += 5 ))` |
| `range(10, 0, -1)` | 10 à 1 | `for (( i = 10; i >= 1; i-- ))` |

`break` sort de la boucle tout de suite, `continue` passe directement au tour suivant :

break_continue.py

```python
for i in range(1, 20):
    if i % 3 == 0:
        continue          # on saute les multiples de 3
    if i > 10:
        break             # et on s'arrête après 10
    print(i, end=" ")
print()
```

Dans le terminal

```
$ python3 break_continue.py
1 2 4 5 7 8 10
```

Le `end=" "` remplace le retour à la ligne de `print` par un espace : tout s'affiche sur une ligne. Le `print()` final, vide, va à la ligne.

## 3. Les listes

Une liste range plusieurs valeurs dans une seule variable, dans l'ordre, entre crochets :

Dans l'interpréteur Python

```
>>> notes = [12, 8, 15, 10]
>>> notes[0]
12
>>> notes[3]
10
>>> notes[-1]
10
>>> len(notes)
4
>>> notes.append(17)
>>> notes
[12, 8, 15, 10, 17]
>>> notes[1] = 9
>>> notes
[12, 9, 15, 10, 17]
>>> 15 in notes
True
>>> notes[10]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

- Les cases sont numérotées **à partir de 0** : `notes[0]` est la première, `notes[-1]` la dernière.
- `len(liste)` donne le nombre d'éléments, `liste.append(x)` ajoute `x` à la fin.
- `x in liste` vaut `True` si `x` est dans la liste.
- Lire une case qui n'existe pas provoque une `IndexError`.

**Parcourir une liste.** `for` marche directement sur une liste, sans `range` :

moyenne.py

```python
notes = [12, 8, 15, 10, 17]

somme = 0
meilleure = notes[0]
for note in notes:
    somme = somme + note
    if note > meilleure:
        meilleure = note

print(f"{len(notes)} notes, moyenne {somme / len(notes)}, meilleure {meilleure}")
```

Dans le terminal

```
$ python3 moyenne.py
5 notes, moyenne 12.4, meilleure 17
```

C'est le patron le plus utile de la semaine : **une variable qu'on prépare avant la boucle, qu'on met à jour à chaque tour, et qu'on lit après**. On l'appelle un accumulateur. Python a des fonctions toutes faites pour ces calculs : `sum(notes)`, `max(notes)`, `min(notes)`, `sorted(notes)`. Mais vous devez savoir les écrire vous-mêmes : c'est exactement le genre de raisonnement qu'on vous demandera en entretien.

**Construire une liste dans une boucle** : on part d'une liste vide `[]` et on fait des `append`.

carres.py

```python
carres = []
for i in range(1, 6):
    carres.append(i * i)
print(carres)
```

Dans le terminal

```
$ python3 carres.py
[1, 4, 9, 16, 25]
```

> **En bash** : `notes=(12 8 15)`, `${notes[0]}`, `${#notes[@]}`, `notes+=(17)`, `for n in "${notes[@]}"`. En Python : `notes = [12, 8, 15]`, `notes[0]`, `len(notes)`, `notes.append(17)`, `for n in notes:`.

## 4. Les chaînes de caractères

Une chaîne se manipule presque comme une liste de caractères :

Dans l'interpréteur Python

```
>>> mot = "Python"
>>> mot[0]
'P'
>>> mot[-1]
'n'
>>> len(mot)
6
>>> mot[0:3]
'Pyt'
>>> mot[2:]
'thon'
>>> mot[::-1]
'nohtyP'
>>> mot.upper()
'PYTHON'
>>> mot.lower()
'python'
>>> "y" in mot
True
>>> "Bonjour Alice".replace("Alice", "Bob")
'Bonjour Bob'
>>> "le chat dort".split()
['le', 'chat', 'dort']
>>> "-".join(["a", "b", "c"])
'a-b-c'
>>> "  bonjour  ".strip()
'bonjour'
```

| Écriture | Résultat |
|---|---|
| `mot[i]` | le caractère numéro `i` (à partir de 0) |
| `mot[debut:fin]` | le morceau de `debut` inclus à `fin` exclu, comme `range` |
| `mot[::-1]` | la chaîne à l'envers |
| `mot.upper()`, `mot.lower()` | en majuscules, en minuscules |
| `x in mot` | `True` si `x` apparaît dans `mot` |
| `mot.replace(a, b)` | remplace chaque `a` par `b` |
| `phrase.split()` | découpe aux espaces, renvoie une liste de mots |
| `"-".join(liste)` | colle les éléments d'une liste avec `-` entre eux |
| `texte.strip()` | retire les espaces et retours à la ligne au début et à la fin |

Une boucle `for` parcourt les caractères un par un :

voyelles_exemple.py

```python
phrase = "Les boucles en Python"
nb = 0
for lettre in phrase.lower():
    if lettre in "aeiouy":
        nb += 1
print(f"{nb} voyelles dans « {phrase} »")
```

Dans le terminal

```
$ python3 voyelles_exemple.py
7 voyelles dans « Les boucles en Python »
```

Une chaîne ne se modifie pas sur place : `mot[0] = "J"` provoque une `TypeError`. On fabrique une nouvelle chaîne à la place : `mot = "J" + mot[1:]`.

> **En bash** : `${#mot}`, `${mot:0:3}`, `${mot^^}`. En Python : `len(mot)`, `mot[0:3]`, `mot.upper()`. Attention, en bash `${mot:2:3}` veut dire « 3 caractères à partir du 2 ». En Python, `mot[2:5]` veut dire « du 2 au 5 exclu ».

## 5. Les fonctions

Une fonction donne un nom à un bloc de code. On la **définit** une fois avec `def`, on l'**appelle** autant de fois qu'on veut.

fonctions.py

```python
def dire_bonjour(prenom):
    print(f"Bonjour {prenom} !")


def carre(x):
    return x * x


def est_pair(n):
    return n % 2 == 0


dire_bonjour("Alice")
dire_bonjour("Bob")
resultat = carre(7) + carre(2)
print(resultat)
print(est_pair(10), est_pair(7))
if est_pair(42):
    print("42 est pair")
```

Dans le terminal

```
$ python3 fonctions.py
Bonjour Alice !
Bonjour Bob !
53
True False
42 est pair
```

- `prenom`, `x`, `n` sont les **paramètres** : des variables qui reçoivent les valeurs données à l'appel.
- `return` **renvoie** une valeur à celui qui a appelé la fonction, et arrête la fonction. Cette valeur peut être rangée dans une variable, utilisée dans un calcul, testée dans un `if`.
- La fonction doit être définie **avant** d'être appelée : on met les `def` en haut du fichier, le programme principal en bas.

**`print` ou `return` ?** C'est la confusion numéro un. `print` affiche pour l'humain, et la fonction ne renvoie rien. `return` donne le résultat au programme, qui en fait ce qu'il veut. Une fonction qui calcule quelque chose doit presque toujours **renvoyer** son résultat :

print_return.py

```python
def double_affiche(x):
    print(x * 2)


def double_renvoie(x):
    return x * 2


a = double_affiche(5)
b = double_renvoie(5)
print("a vaut", a)
print("b vaut", b)
```

Dans le terminal

```
$ python3 print_return.py
10
a vaut None
b vaut 10
```

`double_affiche` a bien affiché 10, mais `a` vaut `None`, la valeur spéciale qui veut dire « rien » : la fonction n'a rien renvoyé.

**Un paramètre peut avoir une valeur par défaut**, utilisée si on ne la donne pas :

defaut.py

```python
def saluer(prenom, salutation="Bonjour"):
    return f"{salutation} {prenom} !"


print(saluer("Alice"))
print(saluer("Bob", "Salut"))
```

Dans le terminal

```
$ python3 defaut.py
Bonjour Alice !
Salut Bob !
```

**Les variables d'une fonction lui sont propres.** Une variable créée dans une fonction n'existe que pendant son exécution, et ne touche pas les variables du même nom à l'extérieur. C'est l'inverse de bash, où tout est global sauf avec `local`.

> **En bash**, `return` ne renvoie qu'un code de sortie entre 0 et 255, et pour récupérer un résultat il fallait `echo` puis `$(ma_fonction)`. En Python, `return` renvoie n'importe quelle valeur : un nombre, un texte, une liste. Et les arguments vont entre parenthèses : `carre(7)`, pas `carre 7`.

## Exercices du mardi

### M1. La table de multiplication NIVEAU 1

Demandez un nombre et affichez sa table de 1 à 10, avec `for` et `range`. Puis modifiez le programme pour afficher **toutes** les tables de 1 à 10, avec deux boucles imbriquées et une ligne vide entre deux tables.

### M2. Le compte à rebours, deux fois NIVEAU 1

Écrivez le compte à rebours de 10 à 1, suivi de « Décollage ! », une fois avec `while`, une fois avec `for` et `range`. Laquelle des deux versions préférez-vous, et pourquoi ?

### M3. La somme de 1 à n NIVEAU 1

Demandez `n` et calculez 1 + 2 + … + n avec une boucle. Vérifiez votre résultat avec la formule de Gauss, `n × (n + 1) / 2`, et affichez si les deux sont égaux. À tester avec 10 (55) et 100 (5050).

### M4. Les statistiques d'une classe NIVEAU 2

À partir de la liste `notes = [12, 8, 15, 10, 17, 6, 13, 9]`, calculez **avec des boucles**, sans `sum`, `min` ni `max` :

- la moyenne, arrondie à deux décimales ;
- la note la plus basse et la plus haute ;
- le nombre de notes au-dessus de la moyenne de la classe.

Puis vérifiez vos résultats avec `sum`, `min` et `max`.

### M5. Compter les voyelles NIVEAU 2

Écrivez une fonction `compter_voyelles(texte)` qui **renvoie** le nombre de voyelles d'un texte, majuscules comprises. Testez-la sur `"Anticonstitutionnellement"` (10 voyelles). Bonus : comptez aussi les voyelles accentuées (`é`, `è`, `à`…).

### M6. Le palindrome NIVEAU 2

Comme en bash : écrivez une fonction `est_palindrome(mot)` qui renvoie `True` si le mot se lit pareil dans les deux sens (`kayak`, `radar`). Faites-le d'abord avec `[::-1]`, puis **sans**, avec une boucle qui compare le premier caractère et le dernier, le deuxième et l'avant-dernier, etc.

Niveau 3 : que la fonction accepte aussi les phrases, en ignorant les majuscules et les espaces : `"Esope reste ici et se repose"` est un palindrome.

### M7. FizzBuzz NIVEAU 2

Affichez les nombres de 1 à 30, mais remplacez les multiples de 3 par `Fizz`, ceux de 5 par `Buzz`, et ceux de 3 **et** de 5 par `FizzBuzz`. C'est un grand classique des entretiens d'embauche : l'ordre des conditions compte.

### M8. La recherche NIVEAU 2

Écrivez une fonction `indice(liste, valeur)` qui renvoie la position de la première occurrence de `valeur` dans la liste, ou `-1` si elle n'y est pas. Sans utiliser `in` ni `.index()` : avec une boucle. Testez avec `indice([4, 8, 15, 16, 23, 42], 16)` (3) et `indice([4, 8, 15], 7)` (-1).

### M9. Les nombres premiers NIVEAU 3

Un nombre premier n'est divisible que par 1 et par lui-même (2, 3, 5, 7, 11…). Écrivez une fonction `est_premier(n)`, puis utilisez-la pour construire la liste de tous les nombres premiers inférieurs à 100. Il y en a 25.

Pour aller plus loin : il suffit de tester les diviseurs jusqu'à la racine carrée de `n`. Pourquoi ?

### M10. La saisie des notes NIVEAU 3

Demandez des notes une par une, jusqu'à ce que l'utilisateur tape `fin`. Rangez-les dans une liste, puis affichez le nombre de notes, la moyenne, la plus basse et la plus haute. Si l'utilisateur tape `fin` tout de suite, affichez « Aucune note » au lieu de planter. Réutilisez vos fonctions de M4 en les rangeant dans des `def`.

## Mémo du mardi

| Je veux… | En bash | En Python |
|---|---|---|
| répéter tant que | `while (( n > 0 )); do … done` | `while n > 0:` + bloc |
| répéter n fois | `for (( i = 0; i < n; i++ ))` | `for i in range(n):` |
| de 1 à 10 | `for i in {1..10}` | `for i in range(1, 11):` |
| sortir, sauter un tour | `break`, `continue` | `break`, `continue` |
| créer une liste | `t=(12 8 15)` | `t = [12, 8, 15]` |
| lire une case, la dernière | `${t[0]}`, `${t[-1]}` | `t[0]`, `t[-1]` |
| taille | `${#t[@]}` | `len(t)` |
| ajouter | `t+=(17)` | `t.append(17)` |
| parcourir | `for x in "${t[@]}"` | `for x in t:` |
| est dans | boucle à la main | `x in t` |
| morceau de chaîne | `${mot:0:3}` | `mot[0:3]` |
| majuscules | `${mot^^}` | `mot.upper()` |
| découper en mots | `read -a mots <<< "$phrase"` | `phrase.split()` |
| définir une fonction | `f() { … }` | `def f(x):` + bloc |
| renvoyer un résultat | `echo` puis `$(f)` | `return valeur` |